THE OPEN SOURCE AUDIT

A Structured Examination of Python

Chosen Software:

Python 3.x — PSF License

Course: Open Source Software | VITyarthi

March 2026

Student Name	Registration Number
Agastya Chitransh Shrivastav	24BCE10818
Slot	**Date of Submission**
F11	2/4/2026

1. Introduction

This report is a comprehensive open source audit of Python, one of the most popular and impactful programming languages used globally. Open source software is based on a revolutionary concept: the internal workings of a program should be accessible to anyone willing to look. The source code is therefore open and accessible for anyone willing to look and learn, as opposed to being locked behind a wall of money or law.

This open source audit has chosen Python as the programming language to be audited based on its tremendous growth and impact on various industries. Python was first developed in 1991 by Guido van Rossum and has remained open source ever since. Over time, Python has become synonymous with machine learning for many technology companies while still being used as a first programming language for many students.

From the perspective of licensing, the Python software is licensed under the Python Software Foundation License, which is an open-source license. This license allows users to use, study, modify, and distribute the software without any legal or financial issues. This has been an important factor in the popularity of the software.

But Python is more than just software. Its history and philosophy offer important insights into the success of the software and its potential for the future. These are important factors in the success of the software, which is part of the open-source movement.

This report is divided into five sections: the history and philosophy of Python; the use of Python in a Linux environment; the use of Python in the open-source movement; the comparison of Python with other commercial software; and five shell scripts.

Part A — Origin and Philosophy

Part A1 — The Problem Python Was Created to Solve

The World Before Python

This report offers an extensive open-source audit of the Python language, which has become one of the most popular and influential programming languages in the world. The open-source software movement has been built on the belief in the open access of the source code. The Python language was created in 1991 by Guido van Rossum. The language has remained open and accessible from the very beginning. The language has evolved over the years to become a powerful language in various areas such as web development, data science, and artificial intelligence. The language remains easy to use for new programmers.

Python is released under the Python Software Foundation License, which is a permissive open-source license. This license allows users to use, modify, and redistribute Python with minimal restrictions. This has been a vital factor in the adoption and development of Python as a programming language. Python is more than just a programming language; it is a philosophy based on collaboration and open-source development. This report will discuss Python's history and philosophy, its use in a Linux environment, its role in open-source development, comparisons to other languages, and examples of shell scripting.

The Limitations of ABC

The main problem with ABC was how limited it was. If you needed something it didn't already support, you couldn't add it. There was no way to include libraries, connect it with other tools, or modify how it worked. Since the source code wasn't open, only a small group of people could make changes or fix issues.
Because of this, once you reached its limits, you couldn't work futher . There was no flexibility or room for improvement.

Guido van Rossum understood this clearly. The ideas behind ABC were strong,simple and readable but its closed and rigid design made it impractical.

Guido Started To Build Something Better

Around Christmas time in 1989, Guido van Rossum began work on a new programming language, mostly as a side project. His vision for this language was simple: he wanted to retain the good qualities of the language he had been using, like readability and simplicity, but eliminate the bad qualities, like the restrictions that made it difficult to use for real projects. He wanted a language that was flexible, something that people could build upon, something they could extend, and something they could actually enjoy using.

He also wanted this language to be open and accessible. Anyone could use it, modify it, and add their own touches. He chose the name Python, inspired by the British comedy group Monty Python, because he did not want programming to be too serious, too intimidating.

By 1991, he had released the first version of this language, 0.9.0, through an internet newsgroup. This language had all the basic features, like functions, exception handling, and data types. What's more, it had no cost, no restrictions, and no strings attached. Anyone could use it, modify it, and add their own touches.

Why Did The Timing Matter So Much

This choice made a huge difference. The reason is that at that time, most of the best programming languages were either expensive, difficult to work with, or difficult to read. Most of the time, you had to make a choice between these three options.

Python stood out because it was all three: it was easy to understand, it was powerful enough to get the job done, and it was completely free to use. Anyone could use it, modify it, and build on it, and this wasn't common back then.

What Openness Actually Did

Python filled that void. Because it was open from the very beginning, people all over the world began contributing to it—adding libraries, improving the language, and instructing others on how to use it. And that is the primary reason why, more than three decades later, it is one of the most popular programming languages.

Nowadays, it is used in all kinds of industries. Companies such as Google use it in machine learning, organizations such as NASA use it in scientific research, and social media sites such as Instagram use it in their backend systems. It is also one of the primary programming languages that new students are exposed to because it is very easy to read and use.

A2. The License — What It Actually Says

What is the PSF License?

The source code for the Python program is distributed under the Python Software Foundation License (also known as the PSF License). This license is maintained by the Python Software Foundation. This is a permissive open-source software license. This license allows users to use the Python program for whatever purpose they wish, change the source code, and also distribute the source code.

The only requirement for the source code to be distributed is the inclusion of the original license. This flexibility and freedom to use the source code in whatever manner the user wants have made the Python program extremely popular.

The Four Freedoms of Free Software

Richard Stallman the guy behind the GNU Project came up with the idea of four essential freedoms that any truly free software should give you:

Freedom 0	You can run it for whatever purpose you want
Freedom 1	You can look at the source code and understand how it works
Freedom 2	You can share copies with other people
Freedom 3	You can modify it and share your modified version

Python's PSF License gives you all four. You can download CPython from GitHub right now, read every single line of code, change it however you want, compile your own version, and give it to someone else. No permission needed, no fees, no restrictions.

Can a Company Take Python and Sell It Without Sharing Changes?

Yes, and this is an important point. The PSF License is what is known as a permissive license. This is what makes it different from other licenses, such as the GPL. Unlike the GPL, there is no requirement for any altered versions of the code to also be distributed as open-source. This is referred to as “copyleft.” This means there is no obligation for the developer to make any changes to the code publicly available.

This has allowed companies to take the source code for Python, alter it, and then integrate it into closed-source products, yet still remain within the terms of the license by keeping the copyright notice and license.

GPL vs MIT/PSF :- What's the Difference?

This difference can be easily explained by the fact that the GNU GPL follows a "share alike" principle. This means that if a programmer creates a new piece of code using the GPL and wants to share it, the new piece of code also has to be shared using the same license.

On the other hand, the MIT License and the PSF License allow programmers to use the code, change it, and share it without many restrictions, even in a closed-source environment. This has been a big advantage for the Python programming language. If the Python programming language had been using the GPL, many companies would have been unable to use it in a closed-source environment. However, the fact that the licensing was easy and open has been a big advantage. When a programmer has to choose a license for his or her open-source or closed-source program, the best licenses to use would be the MIT License and the PSF License. However, the GPL would be best when the programmer wants to share the improvements made to the original piece of code.

Free Beer vs Free Freedom

In the world of open source, there is an important difference between “free as in price” and “free as in rights.” While the first means simply that something is free of charge, the latter means that users have the right to use, modify, and

view the source code without any restrictions. This is what open source really means.

While Python is free in both senses, the true strength of the language is in the freedom that is granted by the Python Software Foundation License. This license allows users free access to the source code, which can be modified according to their requirements. This is extremely important in practice. For example, a researcher using Python for simulation purposes does not have to worry about getting any updates or bug fixes from any vendor. He can simply look at the source code, determine what is causing the problem, and even fix it. This is because Python is an open source.

A3. The Ethics of Open Source

Should All Software Be Open Source?

Therefore, in situations where software has a direct impact on public welfare, such as in the health sector, voting systems, or government systems, the argument for open-source software becomes even more appealing. This is because if such systems use proprietary software, they demand a certain degree of trust in the vendors because the inner workings of the software cannot be verified. However, open-source software would allow experts in the field to verify the software and make it better, thus creating more trust in the system.

However, creating software is a costly and tedious task. Moreover, innovation happens in the hands of companies that use the proprietary model to keep their business alive. Therefore, if the government were to demand open-source software across the board, this might stifle innovation. The best approach would be to strike a balance. While not all software must be open-source software, government-funded software or critical government infrastructure must be open-source software.

Is It Ethical to Profit From Community Labor?

This is where the discussion gets complicated. Large technology firms such as Google, Amazon, and Meta Platforms heavily depend on open-source software such as Python and Linux, which were developed by individual contributors or small teams, often without receiving direct financial compensation. On a positive note, these firms are contributing back to these communities. Google supports Python development, Meta supports large open-source libraries, and Amazon contributes to Linux development. This is good for the open-source community as a whole.

But again, there are many organizations that are heavily dependent on open-source software but are not contributing back to these communities. This is completely within their rights as per the license agreement. This is called freeriding and is ethically questionable. Freeriding is perfectly within their legal rights but goes against the open-source philosophy.

The Red Hat Model A Fair Middle Ground

Red Hat is often cited as a good example of how this should be done correctly. They don't sell the software; they sell the service and reliability that is associated with it. The source code is still open and available, but the service that is provided is what they are selling.

This is because they are providing actual value. They are not just taking something that is already open source and selling it to people. They are providing something of actual value on top of it. This is in comparison to other companies that are just taking open source software and selling it to people without providing anything in return.

Standing on the Shoulders of Giants

This is because Python is based on the C programming language, which in turn is based on decades of theoretical computer science and mathematical research. At the next level up, powerful tools such as TensorFlow have been created using Python. And at the level above that, machine learning models and applications reside. It is worth noting here that much of the software stack is open source software, which enables smooth collaboration between the global developer community.

This is because the concept of “standing on the shoulders of giants” is one which is used to describe the way in which progress is possible because of the work done by previous individuals. And in the case of software development, it is the presence of open source software that enables it. If Python were not an open source software tool, it is likely that organizations would either have to create their own tools or purchase the right to use alternative tools. The fact that it is an open source tool has clearly helped reduce the barriers to innovation significantly.

Does Open Source Kill Original Innovation?

It’s actually the opposite. The Python world has spawned an enormous quantity of new and significant innovation, particularly in AI, data science, and web development often more than many closed systems.
When the basic tools are shared, people don’t need to spend time duplicating what’s already been done. They can spend time creating new ideas on top of what’s already been done. Open source is not about limiting creativity; it’s about increasing access to it and speeding it up.

Part B — Linux Footprint

How to Install Python on Linux

Python can be installed in the following ways based on your system:

* If your system is Debian/Ubuntu-based, you can use the following command:

  `sudo apt install python3 python3-pip`

* If your system is RHEL/Fedora/CentOS-based, you can use the following commands:
* `sudo dnf install python3`
* `sudo yum install python3`
* If your system is Arch Linux-based, you can use the following command:
  * `sudo pacman

Key Directories — Where Does Python Live?

path	Purpose
/usr/bin/python3	The main interpreter binary
/usr/lib/python3.x/	Standard library modules
/usr/lib/python3/dist-packages/	System-wide third-party packages
~/.local/lib/python3.x/site-packages/	User-installed packages via pip
/usr/share/doc/python3/	Documentation

User and Group :- Who Runs Python?

The Python executable is usually owned by the root user. However, it is executed with the permissions of the user. There is no continuously running process of Python. It is executed only when a script or command is invoked. This is to ensure that the Python process does not use any system resources unnecessarily and is executed only when required.

Considering the security point of view, this is an important point. If a Python script is executed with high privileges, it will have the same level of control over the system as the user executing it. Hence, it is considered to be good practice to execute applications, especially those exposed to the internet, with dedicated users with limited permissions.

Service Management — Running Python as a Service

Python in and of itself is not a service; it is an interpreter. However, Python application services, such as a web application using Flask or Django, can be configured to operate as a service.

Once configured, they can be controlled like any other Linux service.

Open up a bash shell and enter the following commands:
sudo systemctl start myapp.service
sudo systemctl stop myapp.service
sudo systemctl status myapp.service

Update Model — How Security Patches Reach You

Security patches for CPython are provided through the package manager for your operating system. Once a bug is found, the Python Software Foundation fixes it, and then it's distributed through normal update channels.

Bash command for installing the update:
sudo apt upgrade python3

This process is usually much quicker and more democratic. Bugs are announced openly, patches are reviewed openly, and patches are distributed to everyone at once, rather than being reserved for particular users or requiring payment.

Part C — The FOSS Ecosystem

What Does Python Depend On?

CPython the main Python interpreter is written in C, and it leans on several other open-source libraries to do its job:-

Library	Purpose
Libffi	Calling C functions from Python.
openssl	Encryption and secure HTTPS connections.
zlib	Data compression.
readline	Interactive terminal input.
sqlite3	Built-in lightweight database.

What Has Python Made Possible?

Because it is an open system, it's easy to build on, which is why it has grown so quickly. A large number of widely used tools have come from it, including:

- NumPy → numerical and scientific computing
- TensorFlow, PyTorch → machine learning frameworks
- Django, Flask → web development frameworks
- Pandas → data analysis and structured data handling
- Jupyter Notebook → research and interactive computing
- Ansible → automation and configuration management
- Python Package Index (PyPI) → repository with hundreds of thousands of packages

Python and the Web — The LAMP Connection

In old systems like the LAMP stack, Python was used instead of PHP. In modern web development, Python is used as a backend.

The stack for a modern web development would be Linux, Nginx, Python with Django/Flask, and PostgreSQL.

Python uses WSGI, which is a standard for how a web application interacts with a server. This ensures smooth integration of all the components.

Community and Governance — Who Runs Python?

It is not owned by a company; it is run by a formalized community:

- Python Software Foundation (PSF) → oversees and supports the project
- Steering Council → elected group responsible for key decisions
- Core Developers → maintain and improve CPython
- Python Enhancement Proposals (PEPs) → system for proposing changes
- PyCon → international conferences for the community

This structure was formalized in 2018 when Guido van Rossum stepped down as leader.

Part D — Open Source vs Proprietary

Dimension	Python	MATLAB
Cost	Completely free	$500-$2,000+ per user per year
Security	Full source available , anyone can use	Closed ,you trust MathWorks entirely
Support	Community, Stack Overflow, official docs	Official paid support with guaranteed response
Freedom to modify	Total fork it, change it, redistribute	Zero ,cannot modify or redistribute
Who's in control	Nonprofit PSF + elected council + global community	Entirely MathWorks, a private company
Overall verdict	Recommended for nearly all use cases; enormous ecosystem and no lock-in	Better only for legacy engineering workflows tied to MATLAB toolboxes

Overall Verdict :- Would I Deploy It?

Currently, Python is an extremely popular programming language in numerous industries, ranging from web development, data science, and machine learning. One of the primary reasons why Python is so popular is because it is able to offer an effective blend of complexity and practicality. This means that programmers can create complex applications without having to worry about the intricacies of the programming syntax. Moreover, the fact that Python is free to use is an additional advantage for programmers.

Another reason why Python is so popular is because it is an open-source programming language. This means that the source code is available for public use. This is beneficial because programmers can always rely on the global community to audit the code. Moreover, the availability of numerous libraries via the Python Package Index (PyPI) means that programmers can always use these libraries without having to create their own applications.

Shell Script Tasks

Script 1:-

```
# Variables: Store student and project information
STUDENT_NAME="Agastya Shrivastav "
SOFTWARE_CHOICE="Python"

#  Collect system information using command substitution
KERNEL=$(uname -r)                  # Get the Linux Kernel  version
USER_NAME=$(whoami)                 # Get the currently logges-in user
UPTIME=$(uptime -p)                 # Get how long the system has been running
DISTRO=$(lsb_release -d | cut -f2)   # Get the Linux distribution name
DATE=$(date '+%d %B %Y %H:%M')       # Get current date and time

#  Display the reprt
echo "=============================================="
echo " OPEN SOURCE AUDIT - $STUDENT_NAME"
echo "=============================================="
echo "Software     : $SOFTWARE_CHOICE"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Uptime       : $UPTIME"
echo "Distro       : $DISTRO"
echo "Date/Time    : $DATE"
echo "OS LICENSE   : GNU General Public License v2"
echo "==============================================="
```

Output:-

```
==============================================
 OPEN SOURCE AUDIT - Agastya Shrivastav
==============================================
Software      : Python
Kernel        : 5.15.0-1074-gcp
User          : user
Uptime        : up 12 hours, 33 minutes
Distro        : Ubuntu 24.04.3 LTS
Date/Time     : 02 April 2026 16:23
OS LICENSE    : GNU General Public License v2
==============================================
```

Script2: -

```
#!/bin/bash
# Set the package name to check
PACKAGE="python3"
# CHECK if the package is installed
if command -v python3 &>/dev/null; then
   echo "$PACKAGE is installed!"
   # Show version, license and summary
   command -v python3
else
   echo "$PACKAGE is NOT installed."
fi
# Philosophy note based on package name
case $PACKAGE in
   python3)  echo "Python: a language built by community, for everyone" ;;
   httpd)    echo "Apache: the web server that built the open internet" ;;
   vlc)      echo "VLC: built by students, plays everything, owned by no one" ;;
   esac*)    echo "An open source tool that stands on community shoulders" ;;
esac
```

Output:-

```
python3 is installed!
/ext/venvs/cocalc/bin/python3
Python: a language built by community, for everyone
```

Script3:-

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Agastya Shrivastav | Course: Open Source Software
# Description: Loops through key directories and reports permissions and size

# List of directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"
# Loop through each directory
```

```
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Get permissions, owner and group
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        # Get directory size
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done
# Check Python's config directory
PYDIR="/usr/lib/python3"
echo ""
echo "Python Config Directory Check:"
if [ -d "$PYDIR" ]; then
    ls -ld $PYDIR
else
    echo "$PYDIR not found"
fi
```

Output:-

```
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 27M
/var/log => Permissions: drwxr-xr-x root root | Size: 33M
/home => Permissions: drwxr-xr-x root root | Size: 236K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 2.4G
/tmp => Permissions: drwxrwxrwt root root | Size: 56K

Python Config Directory Check:
drwxr-xr-x 1 root root 26 Nov 25  2024 /usr/lib/python3
```

Script 4:-

```
#!/bin/bash
# Accept log file and keyword as arguments
LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error' if none provided
# Counter variable
COUNT=0
# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi
# Read file line by line
while IFS= read -r LINE; do
    # Check if line contains the keyword (case insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
# Print summary
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
# Print last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 5:-

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Agastya Shrivastav | Course: Open Source Software

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose a paragraph using $TOOL, $FREEDOM, $BUILD
# and write it to $OUTPUT using echo and >>
echo "" > $OUTPUT
echo "Open Source Manifesto — $(whoami)" >> $OUTPUT
echo "Generated on: $DATE" >> $OUTPUT
echo "-----------------------------------" >> $OUTPUT
echo "I believe in the power of $TOOL — a tool built freely and shared openly." >> $OUTPUT
echo "To me, freedom means $FREEDOM. That is why I choose open source." >> $OUTPUT
echo "One day, I will build $BUILD and share it with the world, freely." >> $OUTPUT
echo "Because knowledge grows when it is shared, not hoarded." >> $OUTPUT
echo "-----------------------------------" >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

References

On Python's History and Philosophy

Van Rossum, G. (1991). *Python 0.9.0 release notes*. Posted to alt.sources newsgroup.

Van Rossum, G. (2009). *The history of Python* [Blog series]. Retrieved from python-history.blogspot.com

Python Software Foundation. (2024). *About Python*. Retrieved from python.org/about

Python Software Foundation. (2024). *PSF license agreement*. Retrieved from python.org/psf/license

On Open Source Philosophy and Licensing

Raymond, E. S. (1999). *The cathedral and the bazaar*. Retrieved from catb.org/~esr/writings/cathedral-bazaar

Open Source Initiative. (2024). *The open source definition*. Retrieved from opensource.org/osd

SPDX License List. (2024). *PSF-2.0 license entry*. Retrieved from spdx.org/licenses/PSF-2.0.html

choosealicense.com. (2024). *MIT license*. Retrieved from choosealicense.com/licenses/mit

On Linux and Shell Scripting

Shotts, W. (2019). *The Linux command line* (2nd ed.). No Starch Press. Retrieved from linuxcommand.org

GNU Project. (2024). *GNU Bash reference manual*. Retrieved from gnu.org/software/bash/manual

Linux Foundation. (2024). *Linux kernel documentation*. Retrieved from kernel.org/doc

On the Python Ecosystem

Python Software Foundation. (2024). *Python Package Index (PyPI)*. Retrieved from pypi.org

NumPy Developers. (2024). *About NumPy*. Retrieved from numpy.org/about

Django Software Foundation. (2024). *Django documentation*. Retrieved from djangoproject.com

On the Ethics of Open Source

Eghbal, N. (2016). *Roads and bridges: The unseen labor behind our digital infrastructure*. Ford Foundation.

Heartbleed.com. (2014). *The Heartbleed bug*. Retrieved from heartbleed.com